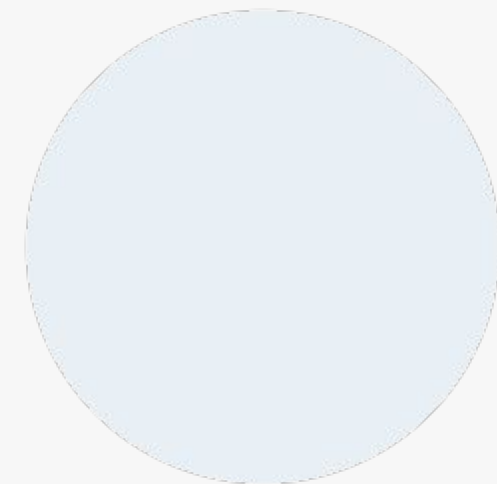


ASTON.



Автоматизация тестирования API



astondevs.ru

RestAssured



Если вашей целью не является досконально протестировать весь API, то проще всего это сделать с REST Assured. Он хорошо подходит для проверки структуры ответов, PVD и smoke-тестов.

Чтобы использовать библиотеку REST-assured, включите в проект зависимость:

```
<dependency>
  <groupId>io.rest-assured</groupId>
  <artifactId>rest-assured</artifactId>
  <version>3.1.0</version>
  <scope>test</scope>
</dependency>
```

Так выглядит простой тест, который будет проверять, что эндпоинт отдает статус 200 OK при обращении к нему:

```
given()
  .baseUrl("http://cookiemonster.com")
  .when()
  .get("/cookies")
  .then()
  .assertThat()
  .statusCode(200);
```

Тестирование добавления — POST



Запись в базу мы добавляем с помощью такого POST-запроса:

`POST /persons`

При этом в теле передается Person в формате JSON:

```
{
  "id": null,
  "name": "Michail"
}
```

В ответ мы должны получить вновь созданного Person в формате JSON и статус 201 (created).

Протестируем и убедимся, что все так и есть:

`@Test`

```
public void whenCreatePerson_thenStatus201() {
```

```
    Person person = new Person("Michail");
```

```
    given().log().body()
```

```
        .contentType("application/json").body(person)
```

```
        .when().post("/persons")
```

```
        .then().log().body()
```

```
        .statusCode(HttpStatus.CREATED.value());
```

```
}
```

given(), when() и then()



Тест REST-assured состоит из трех частей:

- `given()` задает предварительные условия запроса. Здесь это `content-type` (мы задали формат JSON), логирование и тело запроса. Логирование мы включили для учебных целей. Благодаря `log().body()` мы увидим в консоли тело посланного запроса. Тело запроса задано в методе `body(person)`. То есть в теле будет `Person` в JSON-формате. Вообще часть `given()` необязательна, но чаще она есть.
- `when()` — тут мы говорим, что это POST-запрос по адресу `/persons`
- `then()` — тут происходит само тестирование. Мы снова логируем, но уже тело ответа. И проверяем, что код ответа равен 201.

Тестирование редактирования — PUT



Следующий запрос — для редактирования Person:

```
PUT /persons/{id}
```

В теле запроса снова передается Person в формате JSON с заданным id:

```
{  
  "id": 1,  
  "name": "Michail"  
}
```

Тестирование выглядит так (в начале теста мы создаем тестовый Person и берем его id, чтоб отредактировать сущность по адресу /persons/{id}):

```
@Test  
public void whenUpdatePerson_thenStatus200() {  
    long id = createTestPerson("Nick").getId();  
    Person person = new Person("Michail");  
    given().pathParam("id", id).log()  
  
    .body().contentType("application/json").body(person)  
  
    .when().put("/persons/{id}")  
  
    .then().log().body().statusCode(HttpStatus.OK.value())  
        .and().body("name", equalTo("Michail"));  
}
```


Тестирование редактирования — PUT



Тут присутствует метод `pathParam(«id», id)` — на место `id` в пути `/persons/id` мы подставляем реальное значение `id` того `Person`, которого мы хотим отредактировать.

Также мы тестируем тело JSON-ответа — проверяем, что `name` действительно равно `Michail`.

```
@Test
```

```
public void whenUpdatePerson_thenStatus200() {  
    long id = createTestPerson("Nick").getId();  
    Person person = new Person("Michail");  
    given().pathParam("id", id).log()  
  
    .body().contentType("application/json").body(person)  
  
    .when().put("/persons/{id}")  
  
    .then().log().body().statusCode(HttpStatus.OK.value())  
        .and().body("name", equalTo("Michail"));  
}
```

Тестирование удаления — DELETE



Запрос выглядит так:

```
DELETE /persons/{id}
```

В теле запроса ничего не передается.
Возвращается JSON удаленного Person:

```
{  
  "id": 1,  
  "name": "Nick"  
}
```


Тестирование удаления — DELETE



Протестируем. В начале метода мы снова создаем Person, которого будем удалять, и берем его id:

```
@Test
public void
givenPerson_whenDeletePerson_thenStatus200() {
    long id = createTestPerson("Nick").getId();

    given().pathParam("id",
id).log().body().contentType("application/json")

    .when().delete("/persons/{id}")

    .then().log().body()

    .statusCode(HttpStatus.OK.value()).and().body("name",
    equalTo("Nick"));
}
```

Мы логируем тело ответа и проверяем, что его имя равно Nick.

Еще, как обычно, проверяем код ответа.

Тестирование получения — GET



GET /persons

Здесь ничего нового, единственное — метод get():

@Test

public void

givenPerson_whenGetPerson_thenStatus200() {

long id = createTestPerson("Joe").getId();

given().pathParam("id", id)

.when().get("/persons/{id}")

.then().log().body().statusCode(HttpStatus.OK.value())

.and().body("name",

equalTo("Joe"));

}

Тестирование получения списка — GET



```
GET /persons
[
  {
    "id": 1,
    "name": "Joe"
  },
  {
    "id": 2,
    "name": "Jane"
  }
]
```

В начале теста создадим два элемента.

Проверим, что при запросе списка возвращается код ответа 200, и каждый элемент имеет соответствующее имя:

```
@Test
public void
givenPersons_whenGetPersons_thenStatus200() {
    createTestPerson("Joe");
    createTestPerson("Jane");
    when().get("/persons")

        .then().log().body()
        .statusCode(HttpStatus.OK.value())
        .and().body("get(0).name", equalTo("Joe"))
        .and().body("get(1).name", equalTo("Jane"));
}
```

Тестирование получения списка — GET



Обратите внимание, что для получения элементов списка используются команды `get(0)`, `get(1)`.

Ключевые слова `given`, `when` и `then` формируют запрос: `given` определяет, что будет отправлено в запросе, `when` — с каким методом и на какой эндпоинт отправляем запрос, а `then` — как проверяется пришедший ответ. Кроме этого, можно извлечь тело ответа в виде объекта типа `JsonPath` или `XmlPath`, чтобы потом использовать полученные данные.

Реальные тесты обычно больше и сложнее. В запросы добавляются заголовки, куки, авторизация, тело запроса. И если тестируемый API не состоит из десятков уникальных ресурсов, каждый из которых требует особых параметров, вы захотите где-то хранить уже готовые шаблоны, чтобы добавлять их потом к конкретному вызову в тесте.

Для такого в REST Assured существуют:

- `RequestSpecification/ResponseSpecification`;
- базовая конфигурация;
- фильтры.

RequestSpecification и ResponseSpecification



Эти два класса позволяют определить параметры запроса и ожидания от ответа:

```
RequestSpecification requestSpec = given()
```

```
    .baseUrl("http://cookiemonster.com")
```

```
    .header("Language", "en");
```

```
requestSpec.when()
```

```
    .get("/cookiesformonster")
```

```
    .then()
```

```
    .statusCode(200);
```

```
requestSpec.when()
```

```
    .get("/soup")
```

```
    .then()
```

```
    .statusCode(400);
```

```
ResponseSpecification responseSpec = expect()
```

```
    .statusCode(200);
```

```
given()
```

```
    .expect()
```

```
    .spec(responseSpec)
```

```
    .when()
```

```
    .get("/hello");
```

```
given()
```

```
    .expect()
```

```
    .spec(responseSpec)
```

```
    .when()
```

```
    .get("/goodbye");
```

RequestSpecification и ResponseSpecification



Одна спецификация используется в нескольких вызовах, тестах и тестовых классах в зависимости от того, где определена — ограничения нет. Можно даже добавлять несколько спецификаций к одному запросу. Однако это — потенциальный источник проблем:

```
RequestSpecification requestSpec = given()
    .baseUrl("http://cookiemonster.com")
    .header("Language", "en");
RequestSpecification yetAnotherRequestSpec = given()
    .header("Language", "fr");
given()
    .spec(requestSpec)
    .spec(yetAnotherRequestSpec)
    .when()
    .get("/cookies")
    .then()
    .statusCode(200);
```

Лог вызова:

```
Request method:      GET
Request URI:         http://localhost:8080/
Headers:             Language=en
                    Language=fr
                    Accept= */*
Cookies:             <none>
Multiparts:          <none>
Body:                <none>
java.net.ConnectException: Connection refused
(Connection refused)
```


RequestSpecification и ResponseSpecification



Получилось, что в вызов добавлены все заголовки, а вот URI внезапно стал localhost — хотя его добавили в первой спецификации.

Это произошло из-за того, что REST Assured по-разному справляется с переопределениями для параметров запроса (с ответом то же самое). Заголовки или фильтры добавляются в список, а потом по очереди применяются. URI может быть только один, поэтому применяется последний заданный. В последней добавленной спецификации его не задали — поэтому REST Assured переопределяет его дефолтным значением (localhost).

Если добавляете к запросу спецификацию — добавляйте одну.

Совет кажется очевидным, но когда проект с тестами разрастается, возникают классы-хэлперы и базовые тестовые классы, внутри них появляются before-методы. Уследить за тем, что на самом деле происходит с вашим запросом, становится сложно, особенно если тесты пишет сразу несколько человек.

Базовая конфигурация REST Assured



Другой способ шаблонизировать запросы в REST Assured — настроить базовую конфигурацию и определить статические поля класса RestAssured:

`@BeforeMethod`

```
public void configureRestAssured(...) {  
    RestAssured.baseURI = "http://cookiemonster.com";  
    RestAssured.requestSpecification = given()  
        .header("Language", "en");  
    RestAssured.filters(new RequestLoggingFilter(), new  
ResponseLoggingFilter());  
    ...  
}
```

Значения будут автоматически добавляться к запросу каждый раз. Конфигурация сочетается с аннотациями `@BeforeMethod` в TestNG и `@BeforeEach` в JUnit — так можно быть уверенными, что каждый запущенный тест будет начинаться с одними и теми же параметрами.

Тем не менее, конфигурация станет потенциальным источником проблем, ведь она является статической.

Базовая конфигурация REST Assured



Пример: перед каждым тестом берем тестового пользователя, получаем для него авторизационный токен, а потом добавляем его через AuthenticationScheme или авторизационный фильтр к базовой конфигурации. Пока тесты запускаются в один поток, все будет работать. Когда тестов становится слишком много, обычное решение разделить их выполнение на несколько потоков приведет к переписыванию куска кода, чтобы токен из одного треда не попал в соседний.

Фильтры REST Assured



Фильтры изменяют как запросы перед отправкой, так и ответы перед проверкой на соответствие заданным ожиданиям. Пример применения — добавление логирования, или авторизации:

```
public class OAuth2Filter implements AuthFilter {
    String accessToken;
    OAuth2Filter(String accessToken) {
        this.accessToken = accessToken;
    }
    @Override
    public Response filter(FilterableRequestSpecification requestSpec, FilterableResponseSpecification responseSpec, FilterContext ctx) {
        requestSpec.replaceHeader("Authorization", "Bearer " + accessToken);
        return ctx.next(requestSpec, responseSpec);
    }
}

String accessToken = getAccessToken(username, password);
OAuth2Filter auth = new OAuth2Filter(accessToken);
given()
    .filter(auth)
    .filter(new RequestLoggingFilter())
    .filter(new ResponseLoggingFilter())
    ...
```

Фильтры REST Assured



Фильтры, которые добавляются к запросу, хранятся в LinkedList. Перед тем, как сделать запрос, REST Assured изменяет его, проходясь по списку и применяя один фильтр за другим. Потом то же самое делается с пришедшим ответом.

Порядок фильтров имеет значение. Эти два запроса приведут к разным логам: в первом будет указан авторизационный заголовок, во втором — нет. При этом заголовок будет добавлен в оба запроса — просто в первом случае REST Assured сначала добавит авторизацию до того, как залогировать, а во втором — наоборот.

```
given()  
    .filter(auth)  
    .filter(new RequestLoggingFilter())  
    ...  
given()  
    .filter(new RequestLoggingFilter())  
    .filter(auth)
```

Фильтры REST Assured



Помимо обычного правила, что фильтры применяются в том порядке, в котором их добавили, существует еще возможность выставить своему фильтру приоритет, имплементировав интерфейс `OrderedFilter`. Он позволяет выставить особый числовой приоритет фильтру, выше или ниже дефолтного (1000). Фильтры с приоритетом выше будут выполняться раньше обычных, с приоритетом ниже — после них.

Конечно, здесь можно запутаться и случайно выставить двум фильтрам одинаковый приоритет, например, в 999. Тогда первым к запросу будет применен тот, который был добавлен раньше.

Не только фильтры



Как сделать авторизацию через фильтры, показано выше. Но кроме этого способа в REST Assured существует и другой, через AuthenticationScheme:

```
String accessToken = getAccessToken(username, password);
OAuth2Scheme scheme = new OAuth2Scheme();
scheme.setAccessToken(accessToken);
RestAssured.authentication = scheme;
```

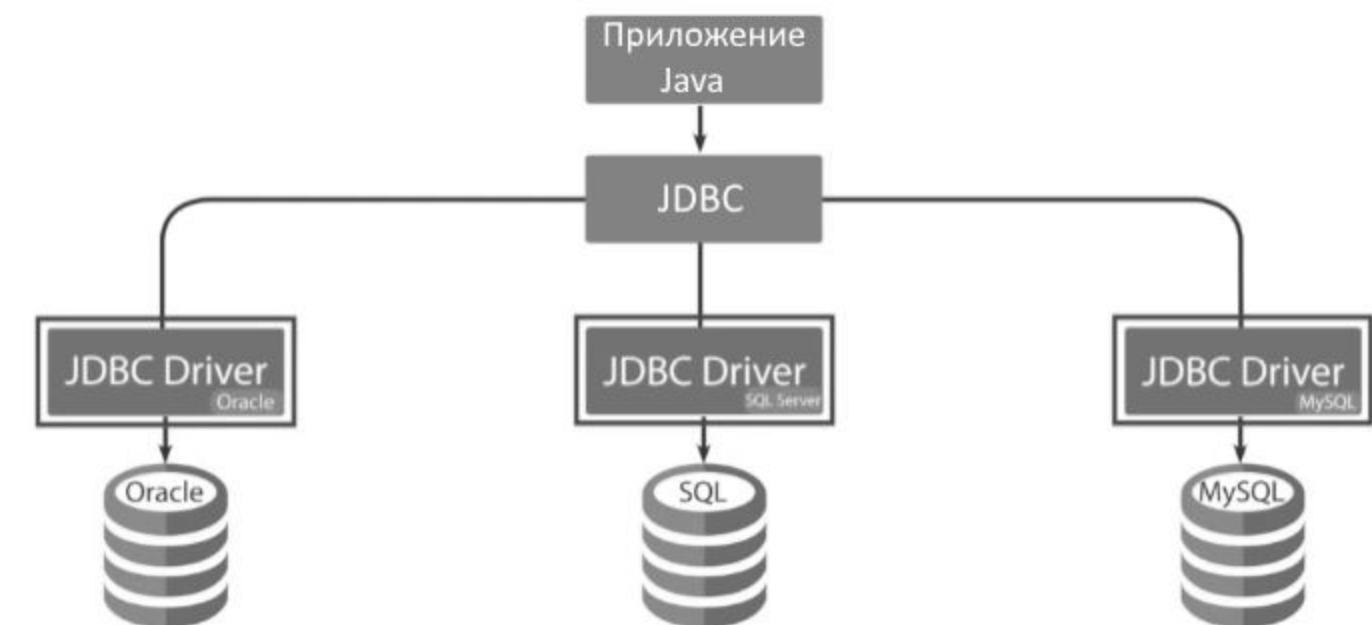
Это устаревший способ. Вместо него стоит выбрать тот, который показан выше.

JDBC



Подключение к базе данных

Чтобы определить единый механизм взаимодействия с СУБД в Java введен специальный прикладной интерфейс API, который называется JDBC. Для работы с определенной СУБД также необходим специальный драйвер. Каждый разработчик определенной СУБД обычно предоставляет свой драйвер для работы с JDBC. То есть если мы хотим работать с MySQL, то нам потребуется специальный драйвер для работы именно MySQL. Как правило, большинство драйверов доступны в свободном доступе на сайтах соответствующих СУБД. Обычно они представляют JAR-файлы. И преимущество JDBC как раз и состоит в том, что мы абстрагируемся от строения конкретной базы данных, а используем унифицированный интерфейс, который един для всех.



JDBC



Для взаимодействия с базой данных через JDBC используются запросы SQL. Чтобы подключить драйвер необходимо добавить соответствующую зависимость в maven pom.xml. В нашем случае это:

```
<dependencies>
  <!--
https://mvnrepository.com/artifact/mysql/mysql-connector-java
  -->
  <dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <version>8.0.26</version>
  </dependency>
</dependencies>
```

Для подключения к базе данных необходимо создать объект `java.sql.Connection`. Для его создания применяется метод:

```
DriverManager.getConnection("jdbc:mysql: //localhost/mysqlDb"
, "root", "password");
```


JDBC



Метод DriverManager.getConnection в качестве параметров принимает адрес источника данных, логин и пароль. В качестве логина и пароля передаются логин и пароль от сервера MySQL. Адрес локальной базы данных MySQL указывается в следующем формате:

```
jdbc:mysql://localhost/<название_базы_данных>
```

После завершения работы с подключением его следует закрыть с помощью метода close():

```
Connection connection =  
    DriverManager.getConnection("jdbc:mysql://localhost/ mysqlDb ",  
    "root", "password");  
  
// работа с базой данных  
connection.close();
```

JDBC



Мы можем определить все данные для подключения непосредственно в программе. Однако что если какие-то данные были изменены потребуется перекомпиляция приложения. Иногда это не всегда удобно, например, отсутствует доступ к исходникам, или перекомпиляция займет довольно продолжительное время. В этом случае мы можем хранить настройки в файле.

Так, создадим в папке программы новый текстовый файл `src/main/resources/database.properties`, в котором определим настройки подключения:

```
url = jdbc:mysql://localhost/mysqlDb
username = root
password = password
```

JDBC



Загрузка параметров в программе будет выглядеть следующим образом:

```
package mysql.connector;

import ...

public class Application {
    public static void main(String[] args) throws SQLException {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver").getDeclaredConstructor().newInstance();
            try (Connection conn = getConnection()){
                System.out.println("Connection to MySQL DB successful!");
            }
        } catch (Exception ex) {
            System.out.println("Connection failed...");
            System.out.println(ex);
        }
    }

    public static Connection getConnection() throws SQLException, IOException {
        Properties props = new Properties();
        try (InputStream in = Files.newInputStream(Paths.get(
            "src/main/resources/database.properties"
        ))) {
            props.load(in);
        }
        String url = props.getProperty("url");
        String username = props.getProperty("username");
        String password = props.getProperty("password");

        return DriverManager.getConnection(url, username, password);
    }
}
```

Взаимодействие с базой данных



Для взаимодействия с базой данных приложение отправляет серверу MySQL команды на языке SQL.

Чтобы выполнить команду, вначале необходимо создать объект

```
Statement statement = conn.createStatement();
```

Для выполнения команд SQL в классе Statement определено три метода:

- **executeUpdate()**: выполняет такие команды, как INSERT, UPDATE, DELETE, CREATE TABLE, DROP TABLE. В качестве результата возвращает int количество строк, затронутых операцией (например, количество добавленных, измененных или удаленных строк), или 0, если ни одна строка не затронута операцией или если команда не изменяет содержимое таблицы (например, команда создания новой таблицы)
- **executeQuery()**: выполняет команду SELECT. Возвращает объект ResultSet, который содержит результаты запроса.
- **execute()**: выполняет любые команды и возвращает значение boolean: true - если команда возвращает набор строк (SELECT), иначе возвращается false.

Метод executeUpdate().



В качестве параметра в него передается собственно команда SQL:

```
int executeUpdate("Команда_SQL")
```

Создадим таблицу и добавим в нее данные:

```
// создание сущности statement
Statement statement = conn.createStatement();
// команда создания таблицы
String sqlCommandCreateTable = "CREATE TABLE products (Id INT PRIMARY
KEY AUTO_INCREMENT, " +
    "ProductName VARCHAR(20), Price INT)";
// создание таблицы
statement.executeUpdate(sqlCommandCreateTable);
// команда добавления данных в таблицу
String sqlCommandAddData = "INSERT Products(ProductName, Price)
VALUES ('iPhone X', 76000), " +
    " ('Galaxy S9', 45000), ('Nokia 9', 36000)";
// добавление данных
int rows = statement.executeUpdate(sqlCommandAddData);
// уменьшение цен на 5000
rows = statement.executeUpdate("UPDATE Products SET Price = Price -
5000");
// удаление 3й записи в таблице
rows = statement.executeUpdate("DELETE FROM Products WHERE Id = 3");
```

Как видно из примеров, не так сложно взаимодействовать с базой данных. Достаточно передать в метод executeUpdate нужную команду SQL.

Метод executeQuery().



Получение данных: Для выборки данных с помощью команды SELECT применяется метод executeQuery:

```
ResultSet executeQuery("Команда_SQL")
```

Метод возвращает объект ResultSet, который содержит все полученные данные. Как эти данные получить?

В объекте ResultSet итератор устанавливается на позиции перед первой строкой. И чтобы переместиться к первой строке (и ко всем последующим) необходимо вызвать метод next(). Пока в наборе ResultSet есть доступные строки, метод next будет возвращать true. Типичное перемещение по набору строк:

```
ResultSet resultSet =  
statement.executeQuery("SELECT * FROM Products");  
while(resultSet.next()) {  
    // получение содержимого строк  
}
```


Метод executeQuery().



После перехода к строке мы можем получить ее содержимое. Для этого у ResultSet определен ряд методов. Некоторые из них:

- getBoolean() возвращает значение boolean
- getDate() возвращает значение Date
- getDouble() возвращает значение double
- getInt() возвращает значение int
- getFloat() возвращает значение float
- getLong() возвращает значение long
- getNString() возвращает значение String
- getString() возвращает значение String

Метод executeQuery().



В зависимости от того, данные какого типа хранятся в том или ином столбце, мы можем использовать тот или иной метод. Каждый из этих методов имеет две версии:

- `int getInt(int columnIndex)`
- `int getInt(String columnLabel)`

Первая версия получает данные из столбца с номером `columnIndex`, индексация столбцов начинается с 1. Вторая версия получает данные из столбца с названием `columnLabel`.

```
ResultSet resultSet =  
statement.executeQuery("SELECT * FROM Products");  
while(resultSet.next()) {  
  
    int id = resultSet.getInt("Id");  
    String name =  
resultSet.getString("ProductName");  
    int price = resultSet.getInt("Price");  
  
    System.out.printf("%d. %s - %d \n", id, name,  
price);  
}
```

Практическое задание



Перейдите в сервис [Postman](#).

Войдите в свой аккаунт и скачайте себе коллекцию Postman Echo.

Написать автотесты для каждого метода из папки Request Methods (проверка тела ответа (просто сравнить значения всех полей) и кода ответа).